

## Task 1 – SHA-256 Hash of ZomatoOrder2024

Linux/Kali/Ubuntu

Run:

```
printf '%s' 'ZomatoOrder2024' | openssl dgst -sha256
```

To save the result:

```
printf '%s' 'ZomatoOrder2024' | openssl dgst -sha256 > zomato_sha256.txt
```

View the file:

```
cat zomato_sha256.txt
```

Expected format:

SHA2-256(stdin)= <64-character-hash>

## Task 2 – Compare MD5 and SHA-256

Create the file:

```
nano my_playlist.txt
```

Example contents:

Blinding Lights  
Shape of You  
Perfect  
Believer  
Night Changes

Save it.

Generate MD5

```
openssl dgst -md5 my_playlist.txt
```

Generate SHA-256

```
openssl dgst -sha256 my_playlist.txt
```

Example output format

MD5:

MD5(my\_playlist.txt)= <32-character-hash>

SHA-256:

SHA2-256(my\_playlist.txt)= <64-character-hash>

Explanation

The hash values are different because MD5 and SHA-256 are different hashing algorithms with different digest sizes and internal mathematical operations.

### Task 3 – Digital Signature Using OpenSSL

Create the file:

```
nano flipkart_order.txt
```

Example:

Flipkart Order

Order ID: FK2024-001

Product: Laptop Bag

Status: Confirmed

Step 1 – Generate a private key

```
openssl genpkey -algorithm RSA -out private_key.pem
```

Step 2 – Generate the public key

```
openssl pkey -in private_key.pem -pubout -out public_key.pem
```

Step 3 – Sign the file

```
openssl dgst -sha256 \
```

```
-sign private_key.pem \
```

```
-out flipkart_order.sig \
```

flipkart\_order.txt

Step 4 – Verify the signature

```
openssl dgst -sha256 \  
-verify public_key.pem \  
-signature flipkart_order.sig \  
flipkart_order.txt
```

Expected:

Verified OK

Test integrity

Change the file:

```
nano flipkart_order.txt
```

Add or modify something.

Run the verification again:

```
openssl dgst -sha256 \  
-verify public_key.pem \  
-signature flipkart_order.sig \  
flipkart_order.txt
```

It should fail because the file is no longer identical to the signed content.

Task 4 – Real-World Hash Collision

Blast-RADIUS – MD5 Collision Attack

A July 2024 report described the Blast-RADIUS attack against the RADIUS authentication protocol. The attack uses an MD5 chosen-prefix collision in a man-in-the-middle scenario to manipulate RADIUS responses.

Researchers demonstrated that an attacker with access to RADIUS traffic could potentially forge an authentication response and obtain unauthorized

network privileges without stealing the user's password. The incident demonstrates why continued dependence on MD5 in security-sensitive protocols creates risks.

Hashing algorithm involved: MD5.

The report was published in July 2024, which falls within the required two-year period for this assignment.